

PART III — BUILDING THE RETRIEVAL PIPELINE

Chapter 13B

Centralized LLM Invocation and Error Handling

“A provider outage is not a design failure. Handling it in nine different places is.”

Chapter 13B — Centralized LLM Invocation and Error Handling

`llm_caller.py` is 913 lines for a project whose actual generation logic fits in a page. Almost none of that length is generation logic. It is a FIFO call gate, an adaptive cooldown derived from live rate-limit headers, a twenty-branch error taxonomy spanning two SDKs and three HTTP libraries, and a recovery path for a specific, once-observed provider failure. Every line of it exists because a single, simpler version broke in production first.

13B.1 Why every LLM call should flow through one wrapper

Chapter 13.7 showed a simplified two-branch version of this idea: classify, then decide whether to retry. The real `llm_caller.py` handles ten distinct Groq exceptions, ten parallel OpenAI-SDK exceptions (because the custom-endpoint and Hugging Face paths speak the OpenAI protocol, not Groq's), plus raw `httpx` and `requests` failures for HTTP-level problems neither SDK wraps. That is roughly thirty except-blocks. Written once, in one file, that is a maintainable taxonomy. Written at every call site that invokes an LLM — and this project has dozens, across drafting, judging, merging, redundancy scanning, and distillation — it is thirty except-blocks multiplied by every call site, each one a chance to classify a failure slightly differently from its neighbor.

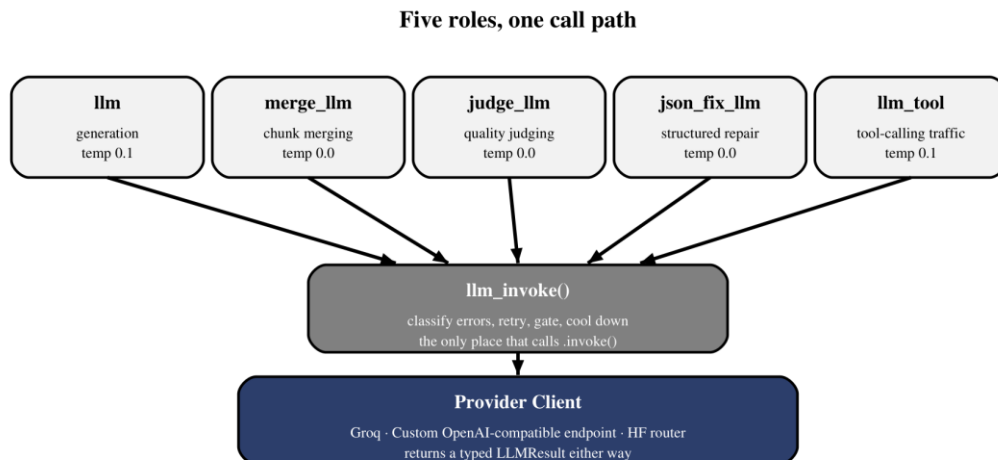


Figure 13B.1 — Every role-specific client still funnels through the same invocation, retry, and classification logic.

Figure 13B.1 is the argument in one picture: five differently-configured clients, one call path. A provider migration, a new retry policy, or a newly discovered failure mode gets fixed once, in `llm_invoke`, and every caller inherits the fix on its next call — not on its next edit.

13B.2 The LLMResult dataclass and the LLMErrorKind enum

Definition — LLMErrorKind

A closed enumeration of failure categories an LLM call can terminate in — rate limiting, authentication, a malformed request, a server error, a network failure, a timeout, and more — used to route every failure through the same typed decision logic instead of a generic exception message.

Every call into `llm_invoke` returns an `LLMResult`, win or lose, rather than raising for failure and returning a string for success. A caller checks `result.ok` exactly once and reads whichever half of the dataclass is populated; nothing downstream needs a `try/except` around a call it did not make.

```
class LLMErrorKind(Enum):
    TOOL_USE_FAILED = auto()      # 400, code="tool_use_failed" - partial gen
    available
    BAD_REQUEST     = auto()      # 400, other cause
    RATE_LIMIT      = auto()      # 429
    AUTH            = auto()      # 401
    PERMISSION      = auto()      # 403
    NOT_FOUND       = auto()      # 404
    UNPROCESSABLE   = auto()      # 422
    SERVER_ERROR     = auto()      # 5xx
    CONNECTION      = auto()      # network failure, no HTTP status
    TIMEOUT         = auto()      # request timed out
    UNKNOWN         = auto()      # anything else

@dataclass
class LLMResult:
    ok: bool
    response: Any = None
    content: str = ""
    error_kind: LLMErrorKind | None = None
    status_code: int | None = None
    error_message: str = ""
    recovered_text: str = ""
    raw_error: BaseException | None = None
```

`recovered_text` is worth noticing before Section 13B.4 explains it: a failed call can still carry partial, usable output. A dataclass that only had `ok` and `content` would have no field to put that in.

13B.3 The Groq error taxonomy

Groq's Python SDK — and, in parallel, the OpenAI SDK used for the custom endpoint and the Hugging Face router — raises a distinct exception class per HTTP status family. `_invoke_once` catches each one by name and maps it to exactly one `LLMErrorKind`, so two SDKs that disagree about class names agree, by the time a caller sees the result, about what actually happened.

Exception	HTTP status	LLMErrorKind
`BadRequestError`	400	`TOOL_USE_FAILED` or `BAD_REQUEST`
`RateLimitError`	429	`RATE_LIMIT`
`AuthenticationError`	401	`AUTH`
`PermissionDeniedError`	403	`PERMISSION`
`NotFoundError`	404	`NOT_FOUND`
`UnprocessableEntityError`	422	`UNPROCESSABLE`
`InternalServerError`	5xx	`SERVER_ERROR`
`APIConnectionError`	— (network)	`CONNECTION`
`APITimeoutError`	— (client-side)	`TIMEOUT`
`APIStatusError` (catch-all)	anything else	`UNKNOWN`

The catch-all matters as much as the specific branches: `APIStatusError` is the parent class every specific Groq exception inherits from, so a status code the taxonomy has not seen yet — Section 13B.17's HTTP 402 is exactly this case — still returns a typed `LLMResult` instead of an uncaught exception. `UNKNOWN` is not a bug in the taxonomy; it is the taxonomy admitting a gap without crashing over it.

13B.4 The `tool_use_failed` recovery path

BUG-F012 is the reason this section exists: Groq once rejected a mid-session tool-call payload with `BadRequestError` / `tool\_use\_failed`, and the pipeline's original response was a hard abort with no attempt to salvage anything. `\_handle\_bad\_request` is the fix — it does not just classify the failure, it reads the error body for a `failed\_generation` field Groq includes specifically for this error code, containing the text the model had generated before the tool-call payload broke.

```

_FUNCTION_SUFFIX_RE = re.compile(r"\s*<function=\w+>\{.*", re.DOTALL)

def _strip_function_suffix(text: str) -> str:
    return _FUNCTION_SUFFIX_RE.sub("", text).strip()
def _handle_bad_request(exc):
    error_detail = exc.response.json().get("error", {})
    if error_detail.get("code") == "tool_use_failed":
        recovered =
    _strip_function_suffix(error_detail.get("failed_generation", ""))
        return LLMResult(ok=False, error_kind=LLMErrorKind.TOOL_USE_FAILED,
                        error_message="tool_use_failed",
                        recovered_text=recovered)
    return LLMResult(ok=False, error_kind=LLMErrorKind.BAD_REQUEST, ...)

```

Common pitfall — Treating a malformed tool call as an empty response

Groq's `failed_generation` field often contains a nearly-complete assistant draft, cut off exactly where the malformed function-call payload begins. Discarding it and returning an empty failure — the original behavior BUG-F012 fixed — throws away real, usable content because the very last part of the response was broken. `_strip_function_suffix` recovers everything before that point instead of nothing.

Whether a caller actually uses `recovered_text` is its own decision — a judge prompt might prefer to fail cleanly and retry, while a best-effort answer path might accept the recovered draft rather than return nothing. The wrapper's job stops at making the recovered text available; what to do with it is a policy decision made above `llm_caller.py`, not inside it.

13B.5 Custom OpenAI-compatible endpoints

`CUSTOM_API_BASE`, `CUSTOM_API_KEY`, and `CUSTOM_API_MODEL_NAME` let `ChatOpenAI` point at any OpenAI-spec server — Together AI, a self-hosted vLLM instance, or a local endpoint — without a code change, the same hot-swap property Chapter 13.2 credited for making the original Groq choice reversible. Getting there took two real, observed bugs.

The first was an ordering bug: a module-level constant read `CUSTOM_API_BASE` before `load_dotenv()` had run inside `main()`, so the client silently fell back to `api.openai.com` and every call failed authentication against the wrong provider with the wrong key. The fix moved credential resolution inside `main()`, immediately after `load_dotenv()`, and added a startup log line — `[LLM] endpoint=... model=...` — specifically so a wrong endpoint is visible before the first failed call, not inferred from it.

The second was a URL-shape mismatch: `.env` held a full endpoint (`.../v1/chat/completions`), which a raw `requests.post` call needs verbatim, but `ChatOpenAI` treats `base_url` as a root and appends `/chat/completions` itself — producing a doubled path and a 404. `_normalize_openai_base_url` strips the trailing `/chat/completions`, `/completions`, or `/responses` suffix before the value reaches `ChatOpenAI`, and does nothing if the value was already a bare root, so it is safe to apply unconditionally rather than gated behind a check for which shape a given `.env` happens to use.

Common pitfall — Reading an env var before load_dotenv() has run

A module-level `os.getenv(...)` executes at import time. If `load_dotenv()` runs later, inside a function, the constant has already captured `None` — and every consumer of that constant is silently wrong for the rest of the process, with no exception to point at the cause. Resolve credentials and endpoints as close as possible to the moment they are used, after configuration is guaranteed loaded, not at module scope.

13B.6 The tolerant HTTP client

A local TGI (Text Generation Inference) server tested against the custom-endpoint path violated the OpenAI tool-call spec in two ways at once: it returned `function.arguments` as a raw JSON object instead of the spec-required JSON-encoded string, and it returned HTTP 500 on multi-turn conversations where an assistant message had an empty `content` field alongside `tool_calls`. Both violations broke LangChain's response parsing before any application code ran, which ruled out fixing them from inside `llm_caller.py` — by the time an exception reached there, the useful information was already gone.

```
def build_tolerant_http_client() -> httpx.Client:
    def _fix_response(response: httpx.Response) -> None:
        # coerce dict-shaped tool arguments into the spec-required JSON
        string
        ...
    def _fix_request(request: httpx.Request) -> None:
        # strip empty `content` fields from multi-turn tool-call messages
        ...
    client = httpx.Client(event_hooks={"response": [_fix_response],
"request": [_fix_request]})
    return client

llm = ChatOpenAI(..., http_client=build_tolerant_http_client())
```

The fix intercepts at the transport layer, before LangChain's Pydantic parsing ever sees the payload — invisible to the rest of the codebase, and applied only when `CUSTOM_API_BASE` actually points at a non-compliant server, so Groq and OpenAI traffic pass through untouched. It is the same lesson as Section 13B.4 from the opposite direction: sometimes the fix belongs even earlier than error classification, at the point where a malformed response first enters the system.

13B.7 The `caller_tag` parameter — grep-friendly trace lines

Every `llm_invoke` call site supplies a `caller_tag` — `QUERY-SIMPLE`, `NAC-MERGE`, `VALIDATE-REDUNDANCY`, `ANSWER-QUALITY` — and every log line `llm_invoke` emits is prefixed with it. The tag costs nothing to add and answers, on sight, a question a stack trace alone cannot: not just that a call failed, but which of the project's dozen distinct LLM roles it was and what it was trying to do.

```
logger.warning(f" [{caller_tag}] 429 received; holding gate, "
                f"token window resets in {delay:.2f}s — retrying at "
                f"front...")
```

A debug log with a thousand lines and no caller tags is a transcript. The same log with tags is an index — `grep NAC-MERGE run.log` isolates one role's entire call history instantly, which is exactly how the HTTP-402 investigation in Section 13B.17 attributed specific failures to specific pipeline stages rather than to "the LLM" in general.

13B.8 Transient versus permanent errors

Not every `LLMErrorKind` deserves the same response. A `RATE_LIMIT` is a timing problem — wait, then the identical request will likely succeed. A `BAD_REQUEST` or `NOT_FOUND` is a correctness problem — the request itself is wrong, and retrying it verbatim will fail identically, forever, while burning latency and retry budget.

LLMErrorKind	Character	Retry?
<code>RATE_LIMIT</code>	Timing — window will refill	Yes — <code>llm_invoke</code> retries automatically
<code>TIMEOUT</code> / <code>CONNECTION</code> / <code>SERVER_ERROR</code>	Transient infrastructure	Worth retrying at the caller's discretion
<code>BAD_REQUEST</code> / <code>NOT_FOUND</code> / <code>UNPROCESSABLE</code>	The request itself is wrong	No — will fail identically every time
<code>AUTH</code> / <code>PERMISSION</code>	Configuration is wrong	No — needs a human, not a retry

In the current implementation, `llm_invoke`'s own retry loop is written specifically for `RATE_LIMIT` — it is the failure mode this project actually hit at scale (Chapter 13.7's 6,000-TPM incident), and it is the one case where the response headers hand back an exact wait time rather than a guess. The other transient kinds return as terminal `LLMResult`'s, leaving the retry decision to whichever caller has the context to make it well — a compression stage might retry once and fall back to the original chunk; a one-shot judge call might not retry at all. Centralizing classification does not require centralizing every retry policy behind it.

13B.9 Why `llm.invoke(...)` is never called directly outside `llm_caller.py`

Every benefit in this chapter — the error taxonomy, the FIFO gate, the adaptive cooldown, the `caller_tag` traceability, the header-hook installation that makes rate-limit awareness possible at all — depends on every call passing through the same function. A single stray `llm.invoke(...)` elsewhere in the codebase would bypass all of it silently: no classification, no gate, no cooldown, and a raised exception instead of a typed result the rest of the pipeline knows how to handle.

This is enforceable as a convention (a code-review rule: no direct `.invoke()` outside `llm_caller.py`) or as a lint rule (forbid importing the raw client type anywhere else). Either way, the invariant is worth protecting deliberately, because the failure mode of violating it is quiet — the stray call site works fine until the exact provider failure the rest of the system was hardened against reaches it first.

13B.10 Multiple LLM roles — one client per job

``llm_setup.py`` constructs five separate ``ChatOpenAI`` (or ``ChatGroq``) instances rather than one shared client reused everywhere, each tuned to what its job actually needs.

Role	Model	Temperature	Job
<code>`llm`</code>	<code>`llama-3.1-8b-instruct`</code>	0.1	Primary generation
<code>`merge_llm`</code>	same as <code>`llm`</code>	0.0	Faithful chunk merging — no creativity
<code>`judge_llm`</code>	<code>`Qwen/Qwen2.5-7B-Instruct`</code>	0.0	Deterministic quality judging
<code>`json_fix_llm`</code>	<code>`Qwen/Qwen2.5-Coder-3B-Instruct`</code>	0.0	Structured-output repair
<code>`llm_tool`</code>	same as <code>`llm`</code>	0.1	Tool-calling traffic, kept separate

Five clients from ``llm_setup.py``, one call path through ``llm_caller.py`` — the roles differ in configuration, never in how their calls are made, classified, or retried. That separation is what let the provider migration in Section 13B.15 move one role at a time without touching the invocation logic at all.

13B.11 Why the answer-generation LLM should not be the answer-quality judge

``judge_llm`` is never the same instance as ``llm``, and the separation is not cosmetic. A model asked to grade its own output carries the same reasoning path, the same blind spots, and the same confident phrasing into the grading pass — a bias problem, not just a redundancy. It is also a calibration problem: self-reported confidence from the model that generated an answer is not evidence the answer is correct, only evidence the model is fluent, and Chapter 20 devotes an entire chapter to exactly this failure.

A dedicated judge running at temperature 0, on its own model instance, at minimum removes the reasoning-path bias and gives the judgment a chance to be reproducible across runs. It is also cheaper in a way that compounds: ``judge_llm`` is a smaller model than ``llm`` in this project's configuration, which Sections 13B.12 and 13B.13 explain is deliberate, not a quality compromise.

13B.12 The dedicated ``json_fix_llm`` — decoupling repair from the primary model

``json_fix_llm`` exists because an earlier design flaw made its necessity obvious. The JSON-repair functions originally accepted an ``llm`` parameter that eight separate call sites dutifully passed — and every one of those values was silently discarded, overwritten by ``llm = json_fix_llm`` on the very next line. The parameter had been dead code since the dedicated repair model was introduced; every caller believed it was choosing a repair model when none of them were.

The fix was not to make the parameter work — no call site had ever needed to diverge from the shared repair model — it was to remove the parameter entirely and make the real architecture visible in the function signature: one dedicated repair model, chosen once, in one place. Decoupling repair calls onto their own instance also protects the primary model's rate-limit budget specifically: a repair call happens only when a structured-output call already failed, which means it is by definition extra load on top of the

pipeline's normal traffic, and routing that extra load onto a separate small model keeps a JSON-repair storm from also starving ``llm``'s token window.

13B.13 Tiered SLM/LLM architecture

Definition — Tiered SLM/LLM architecture

Reserving a large, capable language model for the one task that most needs its reasoning quality — final answer generation — while routing high-frequency, narrowly-scoped tasks like judging and repair to a small language model (SLM) chosen for speed and cost instead.

``json_fix_llm``'s ``Qwen/Qwen2.5-Coder-3B-Instruct`` is a fraction of the size of ``llm``'s ``llama-3.1-8b-instruct`` — deliberately. A JSON-repair call does not need broad reasoning; it needs to reliably reshape malformed text into a schema, a narrow, mechanical task a 3B model handles about as well as an 8B one, at a fraction of the latency and rate-limit cost. Reserve the largest, most expensive model for the task that most needs its judgment — user-facing generation — and let every high-volume, low-judgment task run on the cheapest model that reliably clears the bar.

13B.14 Provider-routing options

Four providers were evaluated for the judge and repair tier specifically, once it became clear Groq had no general-purpose chat model under 8 billion parameters to serve them cheaply.

Provider	Verdict	Why
Groq	Kept for <code>`llm`/`llm_tool`</code>	Fastest inference; no small free model for judge/repair tiers
Custom OpenAI-compatible endpoint	Adopted, then expanded	Full model control; zero per-call cost once self-hosted
Hugging Face Inference Providers router	Adopted, later retired	Free small models; curated catalogue; sustained-load 402s
Google Colab + tunnel	Rejected before and after adoption	Free GPU, but ToS risk and no stable endpoint

Colab is the instructive rejection: every problem was identified **before** any code was written — Colab's terms of service prohibit unattended server-like processes, sessions die after roughly ninety idle minutes, and free-tier GPU availability is not guaranteed. The project wired it in anyway to test the theory, and every anticipated failure mode reproduced empirically: a hardcoded 60-second client timeout racing a 150-second server budget misclassified slow responses as ``UNKNOWN`` instead of ``TIMEOUT``, the FastAPI endpoint caught its own exceptions and returned HTTP 200 with an embedded error field — defeating ``raise_for_status()`` entirely — and the tunnel URL itself changed mid-session, requiring a manual config update. A risk analysis that turns out to be entirely correct once tested is still worth having tested; the alternative was carrying the same theoretical objection into a permanent design decision without ever confirming it.

13B.15 The provider-consolidation timeline

Four architecture decisions, twelve days apart at the extremes, trace a project learning the same lesson twice: split providers to unblock a specific problem, then consolidate once splitting them costs more than it solves.

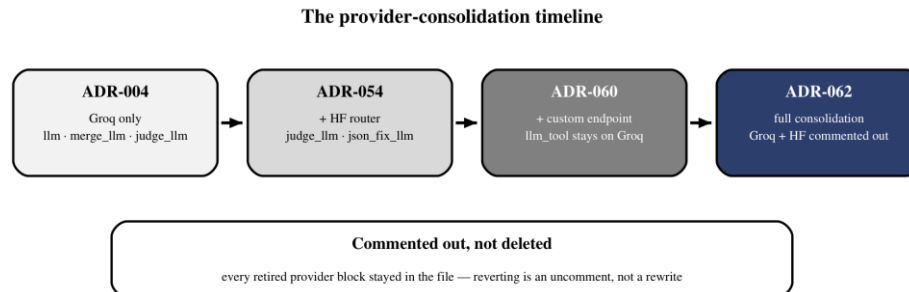


Figure 13B.2 — Each retired provider block was commented out, never deleted — reversal stayed one edit away.

ADR-004 started with Groq alone. ADR-054 added the Hugging Face router for `'judge_llm'` and `'json_fix_llm'` only, once Groq proved to have no small free model for those roles. ADR-060 introduced a self-hosted custom endpoint and moved `'llm'`, `'judge_llm'`, and `'json_fix_llm'` onto it, while `'llm_tool'` stayed on Groq specifically for its tool-calling reliability. ADR-062 finished the consolidation: `'llm_tool'` joined the rest on the custom endpoint, and the Groq and Hugging Face client-construction blocks were commented out in place rather than deleted.

That last detail in Figure 13B.2 is the timeline's real lesson. A migration is a bet, and every bet made in this sequence was reversible by design — commenting out a `'ChatGroq(...)'` block costs one line to undo; deleting it and reconstructing it from memory during a later outage does not. When a benchmark later confirmed the Hugging Face router's sustained-load failures (Section 13B.17), reverting `'judge_llm'` and `'json_fix_llm'` to it was, briefly, an actual step this project took (ADR-061) before the final consolidation — made trivial by exactly this discipline. Walk a consolidation back the moment measurement, not assumption, says the old configuration was better; keep the old code commented, not deleted, so that measurement can be acted on immediately.

13B.16 The ChatGroq + HF-style model path pitfall

`'GEN_MODEL_NAME'` in `'.env'` was set to `'Qwen/Qwen2.5-7B-Instruct'` — a valid, correctly-formatted Hugging Face Hub model path — while feeding `'ChatGroq(model=GEN_MODEL_NAME, ...)'`, a client that only understands Groq's own model catalogue. The result was not a config-validation error; it was a runtime `'HTTP 404 model_not_found'` surfacing deep inside a chunk-merge call, logged as `'[NAC-MERGE] NotFoundError'`.

Common pitfall — A valid model name from the wrong provider's catalogue

The env value was not malformed — it was a real, working model identifier, just for a different provider's client than the one reading it. Two providers sharing one project inevitably share naming conventions in the developer's head even when their catalogues are disjoint; the fix here was mechanical (set `'GEN_MODEL_NAME'` back to a real Groq ID, and keep the invalid value as a commented-out reminder rather than deleting it), but the root cause was structural — one env var feeding two semantically incompatible model catalogues. Namespacing environment variables per provider (`'GROQ_MODEL_NAME'` vs. `'CUSTOM_API_MODEL_NAME'`, never one shared `'MODEL_NAME'`) removes the chance to make this mistake at all.

13B.17 HF Inference Providers Router under sustained load

A five-configuration benchmark — three queries, two runs each, every combination of which roles ran on Groq, the custom endpoint, and the Hugging Face router — surfaced a failure mode invisible in light testing: every setup that routed validator, JSON-fix, or answer-quality traffic through the Hugging Face router logged HTTP 402 responses under sustained load, up to 149 in a single run. The router's free tier returns 402 once request volume or context size exceeds its quota within a rolling window — a distinct failure from the 429 rate-limit path the taxonomy already handled, and one that arrived, at the time, as a generic `'APIStatusError'` rather than its own classified kind.

Setup	HF router traffic	Avg latency (complex query)	Errors
Setup 2	None	6:10	Zero — fastest overall
Setup 1	Judge + JSON-fix + CAQ	31:22	402 on every run
Setup 5	None (all local)	6:58	Zero — close second

The 402s were not merely slow — they were silently corrosive. In the worst-affected log, `'validate_lbc'` returned a genuine model verdict on zero of fourteen calls, defaulting to `'UNKNOWN'` every time, which meant a compression stage's safety judge was effectively absent for the entire run without a single explicit failure being raised anywhere a dashboard would show it. This benchmark, not a theoretical objection, is what motivated ADR-062's full consolidation in Section 13B.15 — 402 density under load was measured, compared directly against a zero-error local configuration, and settled the decision with numbers instead of intuition.

13B.18 Retry with capped exponential backoff

A `'RATE_LIMIT'` result does not retry blindly — the delay grows exponentially with each attempt, capped, and cross-checked against whatever wait time the provider's own response headers report.

```
def _rate_limit_delay(result, attempt, *, base_seconds, max_seconds):
    exponential = min(max_seconds, base_seconds * (2 ** (attempt - 1)))
    header_hint = _groq_wait_seconds(result)    # Groq's own reset-window
    headers
    return max(exponential, header_hint)
```

Constant	Value	Role
<code>'LLM_RATE_LIMIT_MAX_ATTEMPTS'</code>	3	Total attempts before giving up
<code>'LLM_RATE_LIMIT_BACKOFF_BASE_SECONDS'</code>	1.0	First retry's base delay
<code>'LLM_RATE_LIMIT_BACKOFF_MAX_SECONDS'</code>	30.0	Ceiling on the exponential curve
<code>'LLM_RATE_LIMIT_MAX_DELAY_SECONDS'</code>	1800.0	Abort threshold — Section 13B.19

A `'LLM_RATE_LIMIT_BACKOFF_JITTER_SECONDS'` constant exists in configuration but is deliberately unused in `'_rate_limit_delay'` — the code comment is explicit about why: jitter exists to

desynchronize multiple clients retrying in a collision-prone window, but this project's FIFO gate already guarantees only one thread calls the provider at a time, so there is no collision to desynchronize. An unused constant is not always dead code; sometimes it is a documented decision not to apply a general-purpose technique to a specific case where its precondition doesn't hold.

13B.19 The `LLMRateLimitAbortError` — when to stop retrying

Definition — `LLMRateLimitAbortError`

An exception raised when a computed rate-limit backoff delay exceeds a configured maximum wait, converting an indefinitely-postponed retry into an immediate, visible failure instead of a silent multi-hour hang.

A capped exponential backoff still has an edge case: if a provider's own reset window is unusually long — a severe quota exhaustion, not a normal rate-limit blip — the computed delay can exceed any reasonable wait, and retrying anyway just means the pipeline hangs for that entire window before failing regardless. `'LLM_RATE_LIMIT_MAX_DELAY_SECONDS'` (1800 seconds, thirty minutes) is the line: if the required delay would cross it, `'llm_invoke'` releases the FIFO gate — so no other waiting thread is blocked behind a doomed call — and raises immediately instead of sleeping.

```
if delay >= rate_limit_max_delay_seconds:
    _gate_release_to_next()
    raise LLMRateLimitAbortError(delay)
```

This is the same principle as Section 13B.8's transient-versus-permanent distinction, applied one level deeper: even a genuinely transient failure stops being worth waiting for past some threshold, and a system that never defines that threshold will eventually wait the full length of a provider's worst day instead of failing fast and letting a caller — or a human — decide what to do next.

Every mechanism in this chapter exists because a simpler version of `'llm_caller.py'` broke first — a hard abort on a recoverable error, a silently discarded parameter, a config value read one line too early, a free provider's quota limit nobody had classified yet. None of it was designed in advance; all of it was hardened in response to a specific, logged failure. That is the pattern worth carrying forward more than any individual constant or exception class: build the simple wrapper first, and let its real failures — not hypothetical ones — tell you what it needs to become. Chapter 15 leaves single-shot generation behind entirely and asks a harder question: not how to make one LLM call reliable, but how to let a model decide, on its own, when to make another one.